a Real-Time Crypto Dashboard: A Complete Walkthrough

By Admin · July 03, 2026

{# Renders a blog's body. Structured blogs (introduction + typed sections + conclusion) render block-by-block; legacy blogs fall back to the single `content` HTML field. Relies on the .article-body styles in blog_detail.html. #}

Real-time dashboards feel like magic, but the building blocks are simple once you see them laid out. In this walkthrough we'll wire up a live crypto price dashboard step by step.

What we're building

A single-page dashboard that streams live prices for a handful of coins, updates charts in real time, and degrades gracefully when the connection drops.

Tech stack

- Django for the backend and templating
- A websocket feed for live prices
- Chart.js for the live charts
- Redis to fan out updates to connected clients

Connecting to the price feed

Copy

```
import websocket, json

def on_message(ws, message):
    data = json.loads(message)
    print(data['symbol'], data['price'])

ws = websocket.WebSocketApp(
    'wss://stream.example.com/prices',
    on_message=on_message,
)
ws.run_forever()
```

Coins we'll track

Symbol	Name	Update interval
BTC	Bitcoin	1s
ETH	Ethereum	1s
SOL	Solana	2s

Heads up

> *Always throttle chart re-renders. Updating on every single tick will pin the CPU on busy markets — batch updates every 500ms instead.*

Useful references

- Chart.js docs — Official charting library docs
- Django Channels — Websockets for Django

Video walkthrough

A short primer on streaming architectures.

Final result

The finished dashboard with live charts.

Data flow

1. Price feed — External websocket pushes ticks
    - ↳ Validate: Drop malformed messages
2. Fan out via Redis — Broadcast to all connected clients
3. Render chart — Client batches updates every 500ms

Conclusion

That's the whole pipeline — a live feed, a fan-out layer, and a throttled UI. From here you can add more coins, historical charts, or price alerts. Happy building!